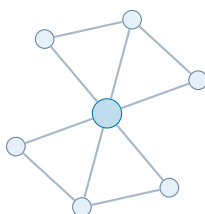


# Détection de communautés dans les graphes sociaux

Étude comparative des algorithmes  
Louvain, Leiden et Graph Neural Networks



Réseaux sociaux · Communautés · Recommandation

Réalisé par  
FERRAH OTHMANE

Encadré par  
Pr. Abdelaziz HESSANE

Année universitaire 2025 – 2026

# Table des matières

|                                                            |           |
|------------------------------------------------------------|-----------|
| <b>Abstract</b>                                            | <b>1</b>  |
| <b>1 Concepts fondamentaux</b>                             | <b>7</b>  |
| 1.1 Fondements de la modélisation en graphes . . . . .     | 7         |
| 1.1.1 Typologie des graphes . . . . .                      | 7         |
| 1.1.1.1 Graphe non orienté . . . . .                       | 7         |
| 1.1.1.2 Graphe orienté . . . . .                           | 7         |
| 1.1.1.3 Graphe pondéré . . . . .                           | 7         |
| 1.1.2 Indicateurs structurels . . . . .                    | 7         |
| 1.2 Les réseaux sociaux vus comme graphes . . . . .        | 8         |
| 1.2.1 Illustrations sur des plateformes réelles . . . . .  | 8         |
| 1.2.2 Propriétés récurrentes des graphes sociaux . . . . . | 8         |
| 1.3 Communautés d'intérêt . . . . .                        | 8         |
| 1.3.1 Critère structurel . . . . .                         | 8         |
| 1.3.2 Nécessité d'algorithmes dédiés . . . . .             | 9         |
| 1.4 Enjeux applicatifs . . . . .                           | 9         |
| 1.4.1 Recommandation et personnalisation . . . . .         | 9         |
| 1.4.2 Marketing ciblé et diffusion virale . . . . .        | 10        |
| 1.4.3 Autres domaines . . . . .                            | 10        |
| <b>2 Données et prétraitement</b>                          | <b>11</b> |
| 2.1 Deux graphes, deux rôles . . . . .                     | 11        |
| 2.2 Le réseau Pokec . . . . .                              | 11        |
| 2.2.1 Contexte et contenu . . . . .                        | 11        |
| 2.2.2 Topologie . . . . .                                  | 11        |
| 2.2.3 Attributs profil . . . . .                           | 12        |
| 2.3 Le réseau YouTube . . . . .                            | 12        |
| 2.3.1 Structure . . . . .                                  | 12        |
| 2.3.2 Intérêt pour l'évaluation . . . . .                  | 12        |
| 2.4 Limites liées à l'échelle . . . . .                    | 12        |
| 2.5 Construction du sous-graphe de travail . . . . .       | 13        |
| 2.5.1 Principe d'échantillonnage . . . . .                 | 13        |
| 2.5.2 Taille retenue . . . . .                             | 13        |
| 2.6 Pipeline de prétraitement . . . . .                    | 13        |
| 2.6.1 Analyse exploratoire . . . . .                       | 14        |

|          |                                                                                      |           |
|----------|--------------------------------------------------------------------------------------|-----------|
| <b>3</b> | <b>La modularité</b>                                                                 | <b>16</b> |
| 3.1      | Critère de qualité d'une partition                                                   | 16        |
| 3.2      | Idée générale                                                                        | 16        |
| 3.3      | Formulation mathématique                                                             | 16        |
| 3.4      | Comment lire la valeur de $Q$                                                        | 17        |
| 3.5      | Illustration sur un graphe-jouet                                                     | 17        |
| 3.6      | Modularité comme objectif d'optimisation                                             | 18        |
| 3.7      | Forces et faiblesses                                                                 | 18        |
| 3.7.1    | Points forts                                                                         | 18        |
| 3.7.2    | Limites connues                                                                      | 19        |
| <b>4</b> | <b>Algorithmes de détection de communautés</b>                                       | <b>20</b> |
| 4.1      | Panorama des approches retenues                                                      | 20        |
| 4.2      | Louvain : regrouper de proche en proche                                              | 20        |
| 4.2.1    | Principe                                                                             | 20        |
| 4.2.2    | Implémentation                                                                       | 20        |
| 4.2.3    | Atouts et limites                                                                    | 21        |
| 4.3      | Leiden : Louvain avec garantie de connexité                                          | 21        |
| 4.3.1    | Correction apportée                                                                  | 21        |
| 4.3.2    | Implémentation et cycle                                                              | 21        |
| 4.4      | GraphSAGE : une autre lecture du graphe                                              | 22        |
| 4.4.1    | Principe                                                                             | 22        |
| 4.4.2    | Positionnement par rapport aux méthodes modulaires                                   | 22        |
| 4.5      | Synthèse comparative                                                                 | 23        |
| <b>5</b> | <b>Implémentation et expérimentation</b>                                             | <b>24</b> |
| 5.1      | Environnement logiciel                                                               | 24        |
| 5.2      | Pipeline KDD                                                                         | 24        |
| 5.3      | Chaîne de traitement des données                                                     | 25        |
| 5.3.1    | Chargement                                                                           | 25        |
| 5.3.2    | Construction et réduction                                                            | 25        |
| 5.4      | Exécution des trois méthodes                                                         | 25        |
| 5.4.1    | Louvain et Leiden                                                                    | 25        |
| 5.4.2    | GraphSAGE (optionnel)                                                                | 26        |
| 5.5      | Protocole d'évaluation                                                               | 26        |
| 5.6      | Déploiement applicatif                                                               | 26        |
| <b>6</b> | <b>Résultats expérimentaux et analyse</b>                                            | <b>28</b> |
| 6.1      | Profil du sous-graphe analysé                                                        | 28        |
| 6.2      | Résultats sur Pokec (40 000 nœuds)                                                   | 28        |
| 6.3      | Validation externe sur YouTube                                                       | 30        |
| 6.4      | Résultats du GraphSAGE                                                               | 31        |
| 6.5      | Balayage du paramètre $\gamma$                                                       | 31        |
| 6.6      | Synthèse et limites                                                                  | 32        |
| <b>7</b> | <b>Application d'un système de recommandation basé sur les communautés détectées</b> | <b>33</b> |
| 7.1      | Du graphe au produit                                                                 | 33        |
| 7.2      | Architecture du pipeline applicatif                                                  | 33        |

|       |                                                     |           |
|-------|-----------------------------------------------------|-----------|
| 7.3   | Recommandation d'amis . . . . .                     | 34        |
| 7.3.1 | Algorithme <code>recommend_friends</code> . . . . . | 34        |
| 7.3.2 | Exemple numérique . . . . .                         | 34        |
| 7.4   | Marketing viral : sélection de seeds . . . . .      | 35        |
| 7.4.1 | Principe et stratégies . . . . .                    | 35        |
| 7.4.2 | Estimation de portée . . . . .                      | 35        |
| 7.5   | Leaders et nœuds-ponts . . . . .                    | 35        |
| 7.6   | Profils communautaires et segmentation . . . . .    | 37        |
| 7.7   | Dashboard interactif . . . . .                      | 37        |
| 7.7.1 | Vue d'ensemble (/) . . . . .                        | 37        |
| 7.7.2 | Recommandation d'amis (/recommend) . . . . .        | 38        |
| 7.7.3 | Marketing viral (/viral) . . . . .                  | 38        |
| 7.8   | Perspectives . . . . .                              | 39        |
|       | <b>Conclusion générale</b>                          | <b>40</b> |

# Table des figures

|     |                                                                                                                  |    |
|-----|------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Ego-réseau : un utilisateur, ses contacts directs et leurs contacts. . . . .                                     | 8  |
| 1.2 | Trois communautés d'intérêt reliées par quelques ponts inter-groupes. . . .                                      | 9  |
| 1.3 | Partition plantée : deux blocs denses séparés par un pont unique. . . . .                                        | 9  |
| 2.1 | Chaîne de réduction appliquée au graphe Pokec. . . . .                                                           | 12 |
| 2.2 | Distribution des degrés sur Pokec (échelle log-log). . . . .                                                     | 14 |
| 2.3 | Tailles des composantes connexes après prétraitement. . . . .                                                    | 14 |
| 2.4 | Répartition du sexe et des principales régions ( <b>kraj</b> ) dans l'échantillon. . .                           | 15 |
| 3.1 | Graphe-jouet à 8 nœuds : partition de référence, sortie Louvain et compa-<br>raison des valeurs de $Q$ . . . . . | 18 |
| 3.2 | Schéma du graphe-jouet : deux $K_4$ reliés par une arête pont. . . . .                                           | 18 |
| 4.1 | Boucle principale de Louvain. . . . .                                                                            | 21 |
| 4.2 | Cas pathologique de Louvain corrigé par le raffinement Leiden. . . . .                                           | 21 |
| 4.3 | Cycle Leiden avec étape de raffinement. . . . .                                                                  | 22 |
| 4.4 | Du voisinage local à l'embedding d'un nœud (GraphSAGE). . . . .                                                  | 22 |
| 4.5 | Aperçu des partitions Louvain et Leiden sur le même sous-graphe Pokec. .                                         | 23 |
| 5.1 | Pipeline KDD implémenté dans le projet. . . . .                                                                  | 25 |
| 5.2 | Du pipeline expérimental aux pages interactives du dashboard. . . . .                                            | 27 |
| 6.1 | Distribution des tailles de communautés Leiden sur le sous-graphe Pokec. .                                       | 28 |
| 6.2 | Visualisation comparative des partitions Louvain et Leiden. . . . .                                              | 29 |
| 6.3 | Cartographie spring-layout des communautés Leiden (extrait visible). . . .                                       | 30 |
| 6.4 | Scores NMI, ARI et AMI sur YouTube. . . . .                                                                      | 31 |
| 6.5 | Projection t-SNE des embeddings GraphSAGE, colorée par cluster. . . . .                                          | 31 |
| 6.6 | Évolution de $k$ et $Q$ en fonction de $\gamma$ (Pokec, 50 000 nœuds). . . . .                                   | 32 |
| 7.1 | Chaîne produit : de la structure du graphe aux applications interactives. .                                      | 33 |
| 7.2 | Logique de recommandation : lien suggéré (pointillé) vers un membre in-<br>fluent de la même communauté. . . . . | 34 |
| 7.3 | Propagation depuis un seed : couverture locale puis extension inter-groupe.                                      | 35 |
| 7.4 | Comparaison PageRank et centralité d'intermédierité par communauté. . .                                          | 36 |
| 7.5 | Leaders d'opinion identifiés par communauté (PageRank). . . . .                                                  | 36 |
| 7.6 | Chaîne de déploiement du pipeline vers le dashboard. . . . .                                                     | 37 |
| 7.7 | Page / du dashboard : vue d'ensemble et KPIs du projet. . . . .                                                  | 38 |
| 7.8 | Page / <b>recommend</b> : recommandation interactive pour l'utilisateur 27 509. .                                | 38 |
| 7.9 | Page / <b>viral</b> : sélection de seeds et estimation de portée. . . . .                                        | 39 |

# Liste des tableaux

|     |                                                                              |    |
|-----|------------------------------------------------------------------------------|----|
| 2.1 | Synthèse des jeux de données mobilisés. . . . .                              | 11 |
| 2.2 | Échelles d'échantillonnage utilisées dans les expériences. . . . .           | 13 |
| 4.1 | Comparaison des trois approches (niveau algorithmique). . . . .              | 23 |
| 5.1 | Stack technique du pipeline expérimental. . . . .                            | 24 |
| 5.2 | Indicateurs mesurés pour chaque méthode. . . . .                             | 26 |
| 5.3 | Principales sorties du pipeline. . . . .                                     | 26 |
| 6.1 | Comparaison Louvain / Leiden sur le sous-graphe Pokec ( $N = 40\,000$ ). . . | 29 |
| 6.2 | Validation YouTube sur le graphe complet. . . . .                            | 30 |
| 6.3 | Guide de choix méthodologique. . . . .                                       | 32 |
| 7.1 | Top-5 recommandations pour l'utilisateur 27509. . . . .                      | 34 |
| 7.2 | Stratégies de sélection des seeds. . . . .                                   | 35 |
| 7.3 | Pages du dashboard et fonctionnalités associées. . . . .                     | 37 |

# Chapitre 1

## Concepts fondamentaux

### 1.1 Fondements de la modélisation en graphes

Les plateformes numériques enregistrent chaque jour des milliards d'interactions entre utilisateurs. Pour les analyser, on les représente le plus souvent sous forme de **graphes** : les acteurs deviennent des *nœuds* (ou sommets) et leurs liens deviennent des *arêtes*. Cette modélisation, issue de la théorie des graphes, constitue le socle de l'analyse des réseaux complexes et de la détection de communautés abordée dans la suite du rapport.

Formellement, un graphe est noté :

$$G = (V, E),$$

où  $V$  désigne l'ensemble des sommets et  $E$  l'ensemble des arêtes. Selon le domaine étudié, un sommet peut représenter une personne, une page web ou une protéine ; une arête traduit une amitié, une citation ou une interaction.

#### 1.1.1 Typologie des graphes

##### 1.1.1.1 Graphe non orienté

Si la relation est réciproque (amitié Facebook, collaboration scientifique), l'arête  $(u, v)$  équivaut à  $(v, u)$ .

##### 1.1.1.2 Graphe orienté

Lorsque la relation possède une direction (abonnement YouTube, relation « suivre » sur X), on distingue une extrémité source et une extrémité cible.

##### 1.1.1.3 Graphe pondéré

Lorsque l'intensité de la relation varie (nombre de messages, fréquence d'échange), chaque arête porte un poids numérique.

#### 1.1.2 Indicateurs structurels

Quelques grandeurs élémentaires guident l'interprétation des algorithmes de partitionnement :

- le **degré** d'un nœud : nombre de voisins directs ;

- un **chemin** : suite de sommets reliés deux à deux ;
- la **distance** : longueur du plus court chemin entre deux nœuds ;
- une **composante connexe** : ensemble maximal de sommets mutuellement accessibles.

## 1.2 Les réseaux sociaux vus comme graphes

Toute plateforme sociale se réduit à deux ingrédients : *qui* est relié à *qui*. Dès lors, l'analyse devient possible avec les outils du graphe.

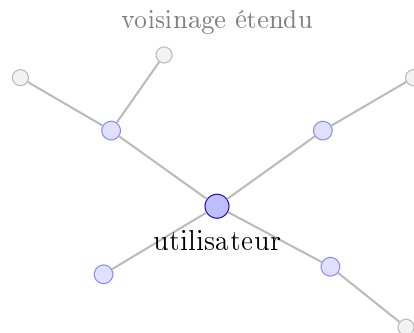


FIGURE 1.1 – Ego-réseau : un utilisateur, ses contacts directs et leurs contacts.

### 1.2.1 Illustrations sur des plateformes réelles

- **WhatsApp** : arête entre contacts ou membres d'un même groupe.
- **YouTube** : nœuds = chaînes ou utilisateurs ; arêtes orientées = abonnements.
- **Facebook** : arêtes non orientées = relations d'amitié.
- **Pokec** : réseau slovaque de référence en recherche, avec profils enrichis (âge, région, sexe) et liens d'amitié.

### 1.2.2 Propriétés récurrentes des graphes sociaux

Les réseaux à grande échelle partagent plusieurs traits distinctifs : taille massive, connectivité locale forte, distribution très inégale des degrés (quelques hubs très connectés) et émergence spontanée de regroupements. Ces propriétés rendent l'exploration manuelle impossible et motivent l'usage d'algorithmes spécialisés.

## 1.3 Communautés d'intérêt

Au-delà de la connectivité globale, les graphes sociaux exhibent une **structure communautaire** : des groupes denses à l'intérieur, reliés faiblement entre eux. Dans la vie quotidienne, on retrouve ce phénomène dans les cercles de joueurs, les réseaux professionnels ou les communautés thématiques ; ces regroupements ne sont pas annotés explicitement, ils se déduisent de la topologie des liens.

### 1.3.1 Critère structurel

Une communauté valide combine :

- une forte densité de liens *intra*-groupe ;
- peu de liens *inter*-groupes.

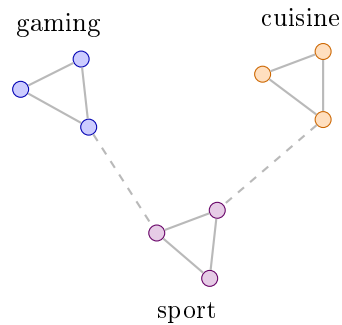


FIGURE 1.2 – Trois communautés d'intérêt reliées par quelques ponts inter-groupes.

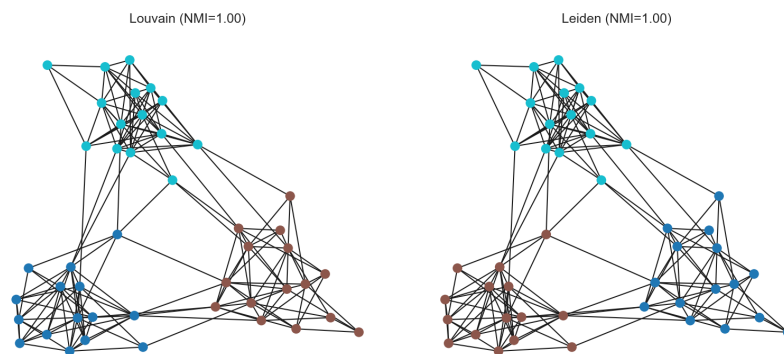


FIGURE 1.3 – Partition plantée : deux blocs denses séparés par un pont unique.

### 1.3.2 Nécessité d'algorithmes dédiés

Sur des graphes réels, l'échelle dépasse rapidement le million de nœuds (YouTube) ou les dizaines de millions d'arêtes (Poké). Trois questions deviennent alors inaccessibles à l'œil nu :

1. quels utilisateurs forment un même groupe ?
2. qui joue un rôle central au sein d'un groupe ?
3. qui assure la liaison entre groupes distincts ?

La détection de communautés relève du **clustering non supervisé sur graphe** : on cherche une partition sans étiquettes préalables, guidée par la structure des connexions.

## 1.4 Enjeux applicatifs

Identifier les communautés ouvre des usages concrets, au cœur de ce projet.

### 1.4.1 Recommandation et personnalisation

Les membres d'un même groupe partagent souvent des préférences proches. La partition permet de suggérer des contacts, des contenus ou des groupes pertinents.

### 1.4.2 Marketing ciblé et diffusion virale

La segmentation par communauté améliore le ciblage publicitaire et l'identification de **leaders d'opinion** ou de **nœuds-ponts** pour lancer une campagne.

### 1.4.3 Autres domaines

La même problématique se retrouve en biologie (réseaux de protéines), en bibliométrie (co-citations) ou en détection de comportements anormaux (groupes suspects dans un réseau financier).

# Chapitre 2

## Données et prétraitement

### 2.1 Deux graphes, deux rôles

Les expériences s'appuient sur deux corpus publics issus de la collection SNAP (Stanford). Ils sont complémentaires : l'un sert de terrain principal pour construire le pipeline, l'autre fournit une vérité de référence pour mesurer la pertinence des partitions détectées.

TABLE 2.1 – Synthèse des jeux de données mobilisés.

|                     | Pokec                                    | YouTube                          |
|---------------------|------------------------------------------|----------------------------------|
| Rôle dans le projet | construction & produit                   | validation externe               |
| Nœuds               | 1 632 803                                | 1 134 890                        |
| Arêtes              | 30 622 564 (orientées)                   | 2 987 624                        |
| Atout principal     | profils utilisateurs (âge, région, sexe) | $\approx 5\,000$ groupes annotés |

Les jeux de données proviennent de la collection SNAP (Stanford) : graphe d'adjacence Pokec, graphe YouTube et partition de référence YouTube. Les identifiants sont anonymisés ; seule la structure topologique et les attributs disponibles sont exploités.

### 2.2 Le réseau Pokec

#### 2.2.1 Contexte et contenu

Pokec est le principal réseau social slovaque. Chaque compte associe des métadonnées (âge, sexe, région administrative **kraj**, centres d'intérêt) à un ensemble de liens d'amitié. L'absence de partition de référence en fait un cas réaliste pour tester un pipeline de bout en bout, depuis la détection jusqu'à la recommandation.

#### 2.2.2 Topologie

Le graphe complet compte plus de 1,6 million de sommets et 30 millions d'arêtes orientées. Après symétrisation (chaque lien d'amitié devient une arête non orientée), la distribution des degrés suit une loi à queue lourde : la majorité des utilisateurs possède peu de contacts, tandis qu'une poignée de hubs concentre un volume important de connexions.

### 2.2.3 Attributs profil

Contrairement à YouTube, Pokec permet d’enrichir l’analyse au-delà de la seule topologie :

- **sexe** : codage binaire SNAP (0 = femme, 1 = homme) ;
- **région** : couple **kraj**, **ville** (ex. *zilinsky kraj*, *zilina*).

Ces variables servent notamment à caractériser les communautés détectées et à vérifier qu’une partition sociale ne coïncide pas mécaniquement avec une variable géographique.

## 2.3 Le réseau YouTube

### 2.3.1 Structure

Le graphe YouTube modélise les liens entre chaînes et utilisateurs de la plateforme. Il reste massif (plus d’un million de nœuds) mais nettement moins dense que Pokec. Environ 39 841 nœuds disposent d’une étiquette de communauté appartenant à l’un des 5 000 groupes fournis par SNAP.

### 2.3.2 Intérêt pour l’évaluation

Les communautés annotées jouent le rôle de *juge externe* : on compare la partition produite par Louvain ou Leiden à la référence à l’aide du NMI (Normalized Mutual Information) et de l’ARI. Les groupes de référence sont majoritairement de petite taille (médiane d’environ 3 membres), ce qui reflète la granularité fine des cercles d’abonnement sur la plateforme.

## 2.4 Limites liées à l’échelle

Traiter les graphes complets sur une machine standard est prohibitif en temps et en mémoire. Le calcul itératif de la modularité, l’entraînement d’un GNN ou même une visualisation globale deviennent rapidement inaccessibles. D’où la nécessité d’un **prétraitement** et d’un **échantillonnage connexe** qui préservent l’essentiel de la structure tout en réduisant l’ordre de grandeur du problème.

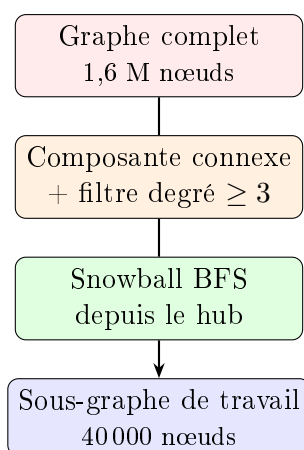


FIGURE 2.1 – Chaîne de réduction appliquée au graphe Pokec.

## 2.5 Construction du sous-graphe de travail

### 2.5.1 Principe d'échantillonnage

On part du nœud de degré maximal (hub) et on explore le voisinage par un parcours BFS (*snowball sampling*) jusqu'à atteindre la taille cible. Cette stratégie garantit un sous-graphe **connexe**, représentatif d'une région dense du réseau, tout en restant reproductible grâce à une graine pseudo-aléatoire fixée.

Listing 2.1 – Échantillonnage BFS depuis le hub (`pokec_bfs_cap`).

```
1 from collections import deque
2
3 def pokec_bfs_cap(g_ig, hub_idx, max_nodes):
4     visited, seen, queue = [], set(), deque([hub_idx])
5     while queue and len(visited) < max_nodes:
6         v = queue.popleft()
7         if v in seen:
8             continue
9         seen.add(v)
10        visited.append(v)
11        for nbr in g_ig.neighbors(v, mode="all"):
12            if nbr not in seen:
13                queue.append(nbr)
14    return visited
```

### 2.5.2 Taille retenue

Le sous-graphe exploité dans les expériences principales contient **40 000 nœuds** et environ **535 000 arêtes** (densité  $\approx 6,7 \times 10^{-4}$ ). Ce compromis permet d'exécuter Louvain, Leiden et le GNN sur un poste de travail classique tout en conservant une structure communautaire riche.

TABLE 2.2 – Échelles d'échantillonnage utilisées dans les expériences.

| Échelle                   | Nœuds  | Usage                                |
|---------------------------|--------|--------------------------------------|
| Exploration démographique | 5 000  | analyse genre/région, premiers tests |
| Montée en charge          | 50 000 | comparaison des temps d'exécution    |
| Analyse principale        | 40 000 | pipeline complet et dashboard        |

## 2.6 Pipeline de prétraitement

Les étapes suivantes sont appliquées avant toute détection de communautés :

1. **Chargement** : lecture des listes d'arêtes ou de la matrice creuse Pokec.
2. **Symétrisation** : conversion en graphe non orienté, agrégation des poids éventuels.
3. **Composante connexe maximale** : retrait des îlots isolés du réseau principal.
4. **Filtrage** : suppression des nœuds de degré inférieur à 3 (réduction du bruit).
5. **Échantillonnage BFS** : extraction du sous-graphe connexe de taille fixée.

6. **Features** : calcul des centralités (degré, PageRank, betweenness, clustering) et normalisation pour le GNN.

Listing 2.2 – Calcul des features structurelles sur le sous-graphe.

```
1 pagerank = nx.pagerank(G_sample)
2 betweenness = nx.betweenness_centrality(G_sample)
3 clustering = nx.clustering(G_sample)
4 degrees = dict(G_sample.degree())
```

Chaque arête est convertie en une paire de sommets adjacents à l'aide de NetworkX. Un contrôle de cohérence (identifiants valides, absence de doublons manifestes) précède la construction du graphe.

### 2.6.1 Analyse exploratoire

Les indicateurs suivants sont calculés sur le sous-graphe retenu : cardinalité de  $V$  et  $E$ , densité, degré moyen et distribution des tailles de composantes connexes. La figure 2.2 confirme le caractère scale-free du réseau ; la figure 2.3 montre la dominance de la composante principale après filtrage ; la figure 2.4 illustre la répartition des attributs profil au sein de l'échantillon.

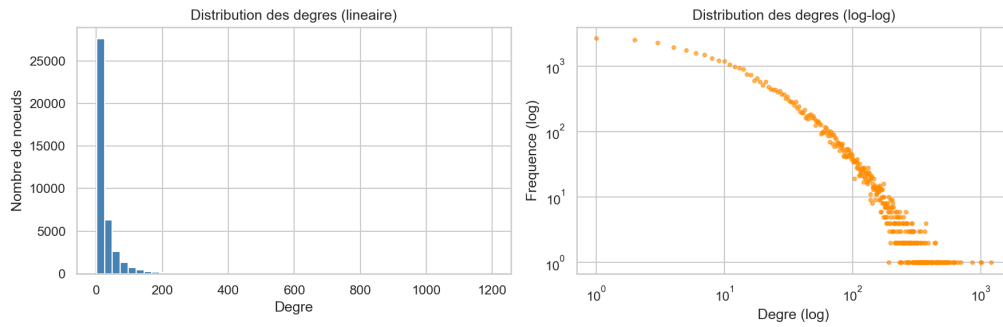


FIGURE 2.2 – Distribution des degrés sur Pokec (échelle log-log).

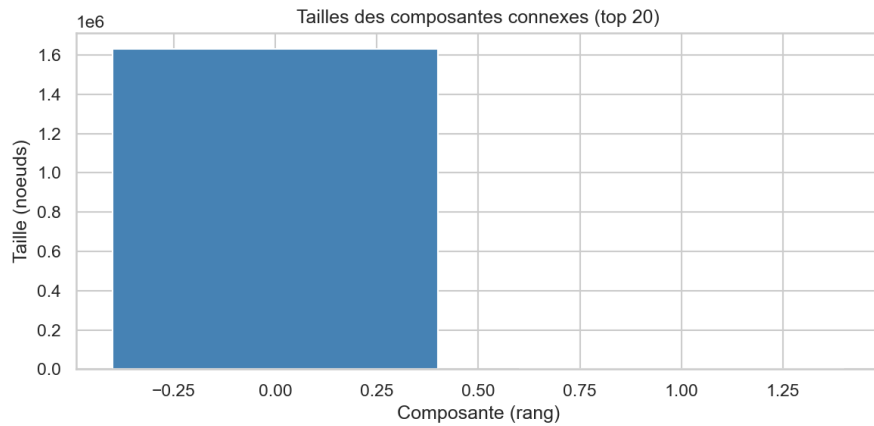


FIGURE 2.3 – Tailles des composantes connexes après prétraitement.

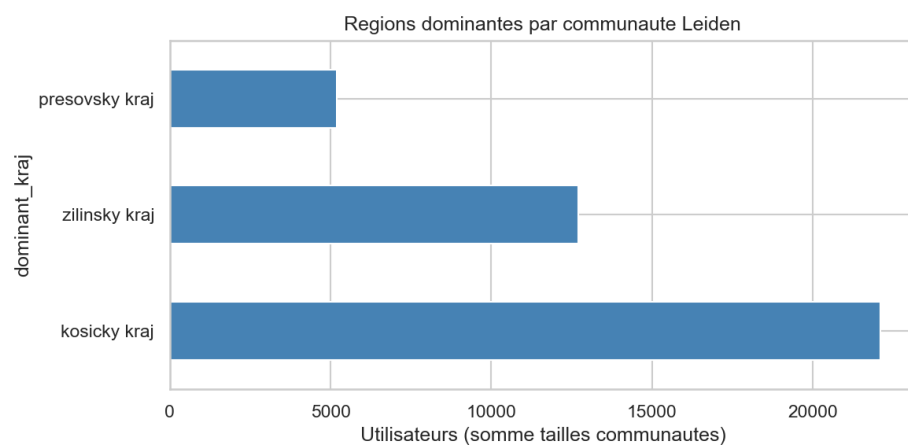


FIGURE 2.4 – Répartition du sexe et des principales régions (**kraj**) dans l'échantillon.

# Chapitre 3

## La modularité

### 3.1 Critère de qualité d'une partition

Une fois le graphe construit (Chapitre 2), il faut juger si le découpage produit par un algorithme est pertinent. Toutes les partitions ne se valent pas : regrouper des nœuds fortement reliés entre eux tout en limitant les liens sortants reflète une organisation sociale crédible ; à l'inverse, répartir les arêtes au hasard entre des groupes arbitraires ne révèle rien sur la structure du réseau.

Le critère recherché combine donc deux propriétés :

- une **densité élevée** de liens à l'intérieur de chaque groupe ;
- une **faible connectivité** entre groupes distincts.

La **modularité**, notée  $Q$ , quantifie précisément cet écart par rapport à un réseau aléatoire de référence. Elle constitue la fonction objectif des algorithmes Louvain et Leiden étudiés au chapitre suivant.

### 3.2 Idée générale

La modularité oppose deux configurations :

1. le graphe *observé*, avec sa structure réelle ;
2. un graphe *aléatoire* possédant les mêmes degrés mais des liaisons placées au hasard.

Si, au sein d'un groupe, les arêtes internes sont nettement plus nombreuses que ce que le hasard produirait, le groupe est jugé significatif. Dans le cas contraire, la partition n'apporte pas d'information structurelle.

Ce raisonnement s'appuie sur le **modèle de configuration** : on conserve le nombre de nœuds et la séquence des degrés ( $k_i$ ), mais on redistribue les arêtes aléatoirement. Ainsi, un réseau dense n'est pas pénalisé à tort : seul l'excès de liens *intra-communautaires* par rapport au hasard est récompensé.

### 3.3 Formulation mathématique

Notons  $A_{ij}$  l'élément de la matrice d'adjacence,  $k_i$  le degré du sommet  $i$ ,  $m$  le nombre total d'arêtes,  $c_i$  la communauté assignée à  $i$ , et  $\delta(c_i, c_j)$  la fonction de Kronecker (valant

1 si  $c_i = c_j$ , 0 sinon). La modularité s'écrit :

$$Q = \frac{1}{2m} \sum_{i,j} \left( A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j).$$

Listing 3.1 – Formule de modularité et évaluation pratique.

```

1 # Q = (1/2m) * sum_{i,j} [A_ij - k_i*k_j/(2m)] * delta(c_i, c_j)
2 Q = community_louvain.modularity(partition, G)

```

Chaque terme de la somme mesure l'écart entre une arête réellement présente ( $A_{ij}$ ) et la probabilité qu'elle existe par hasard ( $k_i k_j / 2m$ ). Le symbole  $\delta$  restreint le calcul aux paires de nœuds appartenant à une même communauté : seuls les liens *internes* contribuent à  $Q$ .

### 3.4 Comment lire la valeur de $Q$

La modularité prend en pratique des valeurs entre 0 et 1 (des cas pathologiques peuvent produire de légères valeurs négatives) :

- $Q \approx 0$  : partition comparable à un découpage aléatoire ;
- $Q \in [0,3; 0,5]$  : structure communautaire modérée, fréquente sur les réseaux sociaux réels ;
- $Q > 0,5$  : séparation nette entre les groupes ;
- $Q \rightarrow 1$  : cas théorique de séparation quasi parfaite, rarement observé.

Dans la littérature, le seuil  $Q > 0,3$  est souvent retenu comme indicateur d'une structure communautaire réellement présente.

### 3.5 Illustration sur un graphe-jouet

Avant de traiter Pokec, le comportement de la métrique est vérifié sur un graphe élémentaire : deux cliques de quatre nœuds ( $K_4$ ) reliées par une unique arête *pont*. La partition attendue regroupe chaque clique séparément.

- **Partition correcte** (6 liens internes, 1 externe par groupe) :  $Q$  élevé ( $\approx 0,42$  sur notre exemple).
- **Partition incorrecte** (coupe une clique en deux) :  $Q$  chute nettement.

Louvain retrouve alors la partition vraie avec NMI = 1,0, ce qui valide l'implémentation avant passage à l'échelle.

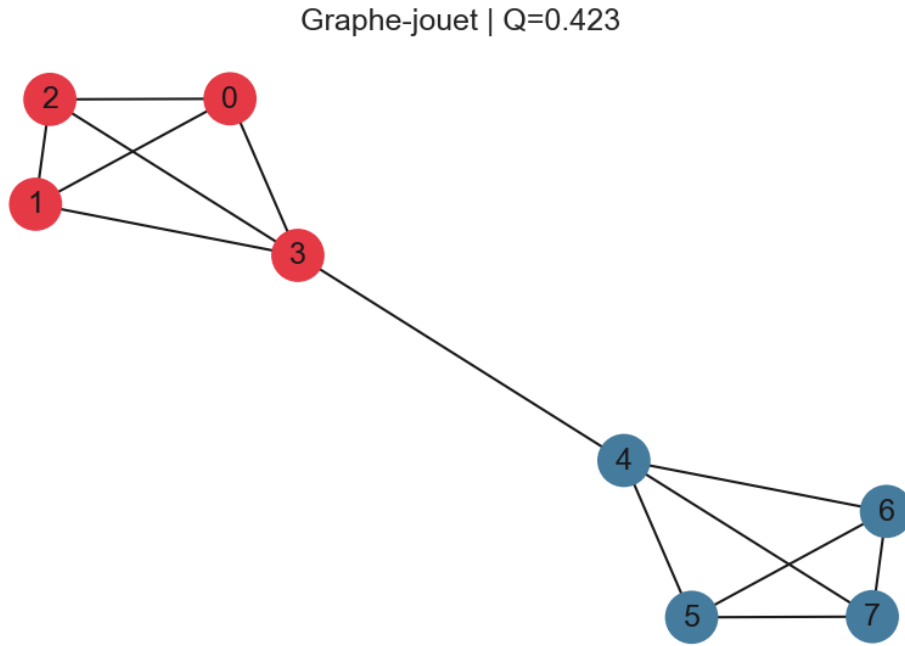


FIGURE 3.1 – Graphe-jouet à 8 nœuds : partition de référence, sortie Louvain et comparaison des valeurs de  $Q$ .

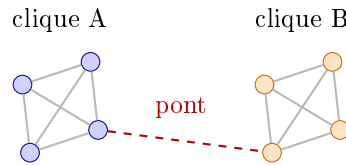


FIGURE 3.2 – Schéma du graphe-jouet : deux  $K_4$  reliés par une arête pont.

## 3.6 Modularité comme objectif d'optimisation

Louvain et Leiden résolvent le même problème : trouver une partition  $\mathcal{C}$  qui maximise

$$\max_{\mathcal{C}} Q.$$

Louvain procède de manière gloutonne (déplacements locaux puis agrégation); Leiden ajoute une phase de raffinement garantissant des communautés connexes. Dans les deux cas,  $Q$  guide chaque décision de fusion ou de déplacement de nœud.

## 3.7 Forces et faiblesses

### 3.7.1 Points forts

- interprétation directe (plus  $Q$  est grand, meilleure est la séparation);
- aucune étiquette préalable requise (apprentissage non supervisé);
- calculable sur des graphes à plusieurs millions de sommets;
- base théorique commune à Louvain, Leiden et de nombreuses variantes.

### 3.7.2 Limites connues

- **NP-difficile** : l'optimum global est introuvable en temps polynomial ; seules des approximations gloutonnes sont utilisées.
- **Limite de résolution** (*resolution limit*) : de petites communautés peuvent être absorbées dans de plus grandes.
- **Sensibilité à la densité globale** : sur des graphes très denses ou très clairsemés,  $Q$  peut mal refléter la qualité perçue.

Ces contraintes motivent l'étude d'approches complémentaires (Leiden pour la connexité, GNN pour l'exploitation d'attributs) développées dans les chapitres suivants.

# Chapitre 4

## Algorithmes de détection de communautés

### 4.1 Panorama des approches retenues

La modularité  $Q$  (Chapitre 3) sert de fil conducteur pour comparer trois approches implémentées dans ce projet :

- **Louvain** (2008) : heuristique gloutonne rapide, référence historique ;
- **Leiden** (2019) : variante corrigeant les communautés déconnectées ;
- **GraphSAGE** (GNN) : apprentissage d’embeddings puis regroupement (K-Means).

Les deux premières maximisent  $Q$  ; le GNN optimise la prédiction de liens et ne cible pas directement la modularité.

### 4.2 Louvain : regrouper de proche en proche

#### 4.2.1 Principe

**Louvain** [2] part d’une partition fine (chaque nœud dans sa propre communauté), puis améliore  $Q$  par déplacements locaux : chaque sommet rejoint la communauté voisine qui maximise le gain de modularité. Une fois la phase stabilisée, chaque communauté est contractée en un *super-nœud* et le processus recommence sur le graphe réduit.

**Analogie** : dans une salle, chacun rejoint le groupe où il a le plus d’amis ; ensuite chaque groupe est traité comme une seule personne et l’opération est répétée.

#### 4.2.2 Implémentation

Bibliothèque `python-louvain` : `community.best_partition(G, resolution= $\gamma$ )`. Le paramètre de résolution  $\gamma$  module la granularité ( $\gamma = 1$  par défaut).

Listing 4.1 – Partition Louvain et calcul de  $Q$ .

```
1 partition_louvain = community_louvain.best_partition(  
2     G_sample, resolution=1.0, random_state=RANDOM_SEED  
3 )  
4 Q_louvain = community_louvain.modularity(partition_louvain,  
     G_sample)
```

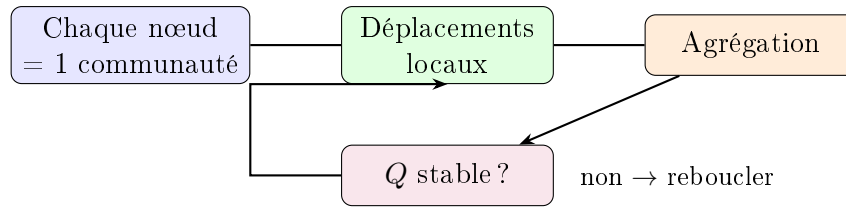


FIGURE 4.1 – Boucle principale de Louvain.

### 4.2.3 Atouts et limites

**Atouts** : simplicité, rapidité sur graphes massifs, large adoption dans la littérature.

**Limites** :

- optimum *local* (pas de retour arrière) ;
- communautés parfois **déconnectées** (un nœud-pont déplacé peut scinder un groupe) ;
- résultats légèrement sensibles à l'ordre de parcours des nœuds.

## 4.3 Leiden : Louvain avec garantie de connexité

### 4.3.1 Correction apportée

**Leiden** [3] conserve l'objectif  $\max Q$  mais insère une phase de **raffinement** entre le déplacement local et l'agrégation : chaque communauté est ré-examinée, les sous-groupes faiblement reliés sont scindés avant contraction. Toute communauté produite est ainsi **connexe**, ce que Louvain ne garantit pas.

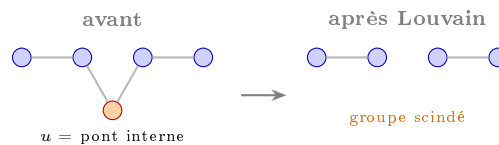


FIGURE 4.2 – Cas pathologique de Louvain corrigé par le raffinement Leiden.

### 4.3.2 Implémentation et cycle

Bibliothèque `leidenalg` avec `ModularityVertexPartition` (ou `RBConfigurationVertexPartition` pour  $\gamma \neq 1$ ).

Listing 4.2 – Partition Leiden et modularité associée.

```

1 partition_leiden = leidenalg.find_partition(
2     G_ig_sample, leidenalg.ModularityVertexPartition,
3     seed=RANDOM_SEED, n_iterations=15
4 )
5 Q_leiden = partition_leiden.modularity

```

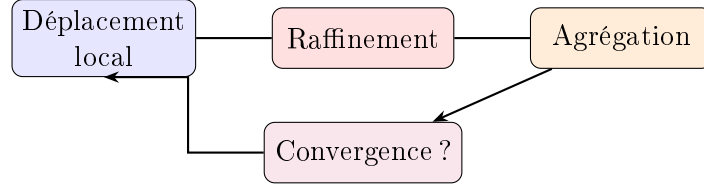


FIGURE 4.3 – Cycle Leiden avec étape de raffinement.

Sur les grands sous-graphes Pokec, Leiden atteint en pratique une modularité comparable ou supérieure avec un temps d'exécution nettement inférieur à Louvain (jusqu'à un facteur 7 sur 50 000 nœuds dans nos essais).

## 4.4 GraphSAGE : une autre lecture du graphe

### 4.4.1 Principe

Les **Graph Neural Networks** n'optimisent pas  $Q$ . Ils apprennent, pour chaque nœud, un vecteur (*embedding*) qui résume son voisinage via un mécanisme de **message passing** :

$$h_v^{(k+1)} = f(h_v^{(k)}, \text{AGG}(\{h_u^{(k)} : u \in \mathcal{N}(v)\})).$$

Dans notre pipeline, **GraphSAGE** est entraîné sur une tâche de *link prediction* à partir de quatre features structurelles (degré, PageRank, betweenness, coefficient de clustering). Les embeddings obtenus sont ensuite regroupés par K-Means pour produire une partition.

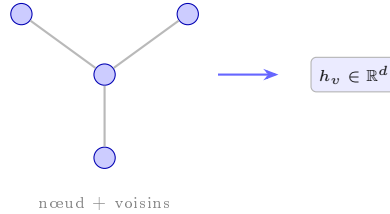


FIGURE 4.4 – Du voisinage local à l'embedding d'un nœud (GraphSAGE).

### 4.4.2 Positionnement par rapport aux méthodes modulaires

Le GNN excelle lorsque des attributs riches (texte, intérêts, contenus) sont disponibles. Dans notre configuration minimale (centralités seules, pas de supervision communautaire), la modularité obtenue après K-Means reste nettement inférieure à Louvain/Leiden ( $Q \approx 0,10$  contre  $Q \approx 0,55$  sur 40 000 nœuds). Les embeddings demeurent néanmoins utiles pour la visualisation (projection t-SNE, Chapitre 6).

## 4.5 Synthèse comparative

TABLE 4.1 – Comparaison des trois approches (niveau algorithmique).

|                      | Louvain        | Leiden    | GNN (GraphSAGE)      |
|----------------------|----------------|-----------|----------------------|
| Objectif             | max $Q$        | max $Q$   | prédiction de liens  |
| Attributs de nœuds   | non            | non       | oui (features)       |
| Communautés connexes | non garanti    | garanti   | dépend du clustering |
| Complexité           | faible         | moyenne   | élevée               |
| Implémentation       | python-louvain | leidenalg | PyTorch Geometric    |

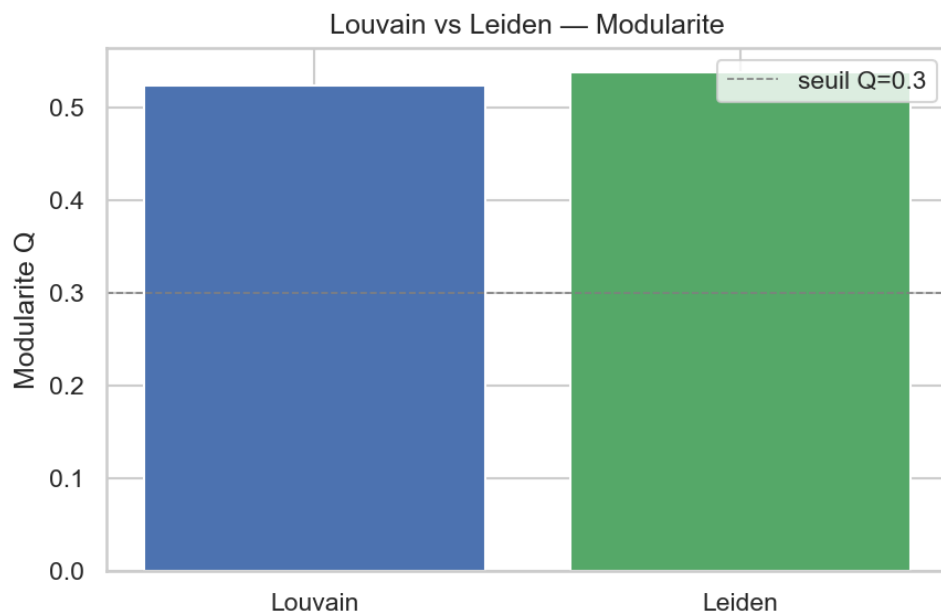


FIGURE 4.5 – Aperçu des partitions Louvain et Leiden sur le même sous-graphe Pokec.

**Choix retenu pour le produit :** Leiden alimente la partition de référence (meilleur compromis  $Q$  / temps / connexité). Louvain sert de baseline rapide. Le GNN est exploré comme piste complémentaire lorsque des attributs plus riches seront disponibles. Les mesures quantitatives détaillées sont reportées au Chapitre 6.

# Chapitre 5

## Implémentation et expérimentation

### 5.1 Environnement logiciel

L'ensemble du projet est développé en **Python 3**. Le tableau 5.1 résume les bibliothèques clés.

TABLE 5.1 – Stack technique du pipeline expérimental.

| Bibliothèque                | Rôle                                                |
|-----------------------------|-----------------------------------------------------|
| NetworkX                    | construction du graphe, centralités, visualisations |
| python-louvain              | partition Louvain, calcul de $Q$                    |
| leidenalg                   | partition Leiden (communautés connexes)             |
| PyTorch + PyTorch Geometric | GraphSAGE, link prediction                          |
| scikit-learn                | K-Means sur embeddings, NMI/ARI                     |
| igraph / SciPy              | chargement matrice creuse Pokec                     |

### 5.2 Pipeline KDD

Le travail suit les six étapes du processus KDD [1], de la donnée brute au déploiement applicatif :

1. **Sélection** : chargement des graphes Pokec et YouTube.
2. **Prétraitement** : symétrisation, composante connexe maximale, filtre degré  $\geq 3$ , échantillonnage BFS.
3. **Transformation** : centralités (degré, PageRank, betweenness, clustering).
4. **Fouille** : Louvain, Leiden, GraphSAGE (optionnel via `RUN_GNN`).
5. **Évaluation** : modularité  $Q$ , NMI, ARI, temps d'exécution, connexité.
6. **Déploiement** : dashboard Next.js interactif.

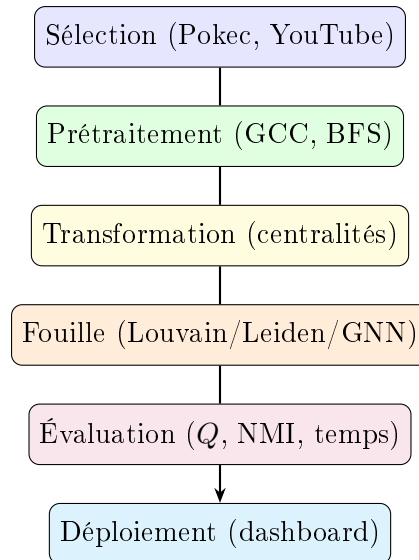


FIGURE 5.1 – Pipeline KDD implémenté dans le projet.

## 5.3 Chaîne de traitement des données

### 5.3.1 Chargement

Les listes d’arêtes sont lues ligne par ligne (identifiant source, identifiant cible). Exemple de format :

```

1 25
1 36
2 45

```

Pokec est également chargeable via `scipy.io.mmread` (matrice creuse  $\approx 400$  Mo).

### 5.3.2 Construction et réduction

Chaque ligne devient une arête NetworkX. Le graphe orienté Pokec est symétrisé, réduit à sa composante connexe maximale, filtré (degré  $\geq 3$ ), puis échantillonné par BFS depuis le hub jusqu’à **40 000 nœuds** ( $\approx 535\,000$  arêtes). Les mêmes opérations préparatoires sont appliquées à YouTube pour la validation externe.

## 5.4 Exécution des trois méthodes

Les algorithmes du Chapitre 4 sont invoqués sur **le même sous-graphe** afin de garantir une comparaison équitable.

### 5.4.1 Louvain et Leiden

- `community.best_partition(G, resolution= $\gamma$ )` avec  $\gamma = 1$  par défaut ;
- `leidenalg.find_partition(G, RBConfigurationVertexPartition)` pour Leiden ;

- enregistrement de  $Q$ , du nombre de communautés  $k$ , du temps CPU et du taux de communautés déconnectées.

Un **balayage de  $\gamma$**  ( $\gamma \in [0,4; 1,6]$ ) explore la sensibilité de  $k$  et  $Q$ ;  $\gamma = 1$  sert de référence standard.

### 5.4.2 GraphSAGE (optionnel)

Si `RUN_GNN = True` :

1. normalisation z-score des quatre features structurelles ;
2. entraînement GraphSAGE (link prediction, split 80/10/10) ;
3. clustering K-Means ( $k$  aligné sur Leiden) pour obtenir une partition.

## 5.5 Protocole d'évaluation

TABLE 5.2 – Indicateurs mesurés pour chaque méthode.

| Indicateur                | Interprétation                                                 |
|---------------------------|----------------------------------------------------------------|
| Modularité $Q$            | qualité interne (sans vérité terrain)                          |
| Nombre $k$ de communautés | granularité de la partition                                    |
| Temps d'exécution         | coût computationnel                                            |
| NMI / ARI                 | accord avec YouTube (ground truth)                             |
| Connexité                 | fraction de communautés déconnectées (Leiden $\rightarrow 0$ ) |

Sur Pokec, l'accord entre Louvain et Leiden est mesuré par le NMI mutuel ( $\approx 0,74$  sur 40 000 nœuds). Sur YouTube, le NMI face aux groupes SNAP sert de validation externe ( $\approx 0,54$  pour Leiden).

## 5.6 Déploiement applicatif

Les résultats intermédiaires du pipeline — partitions, centralités, profils communautaires et scénarios de recommandation — sont structurés pour alimenter le dashboard interactif. Le tableau 5.3 résume les principales sorties produites.

TABLE 5.3 – Principales sorties du pipeline.

| Sortie                      | Contenu                                                |
|-----------------------------|--------------------------------------------------------|
| Partitions Louvain / Leiden | étiquettes communautaires par nœud                     |
| Features structurelles      | centralités (degré, PageRank, betweenness, clustering) |
| Profil communautaires       | agrégats genre et région par cluster                   |
| Métriques YouTube           | scores NMI et ARI de validation externe                |
| Leaders et ponts            | nœuds influents et nœuds-ponts par communauté          |
| Scénarios applicatifs       | recommandations d'amis et seeds marketing viral        |
| Embeddings GNN              | représentations vectorielles des nœuds                 |

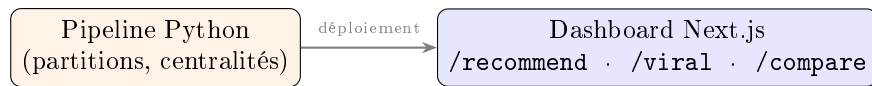


FIGURE 5.2 – Du pipeline expérimental aux pages interactives du dashboard.

Les résultats quantitatifs détaillés (tableaux, visualisations) sont analysés au Chapitre 6 ; l'exploitation applicative (recommandation, marketing viral) au Chapitre 7.

# Chapitre 6

## Résultats expérimentaux et analyse

### 6.1 Profil du sous-graphe analysé

Les expériences portent sur un extrait connexe de Pokec (Chapitre 2) : **40 000 nœuds**, **535 654 arêtes**, densité  $6,7 \times 10^{-4}$ . La distribution des degrés suit une loi à queue lourde et la partition Leiden produit une diversité de tailles de communautés (figure 6.1).

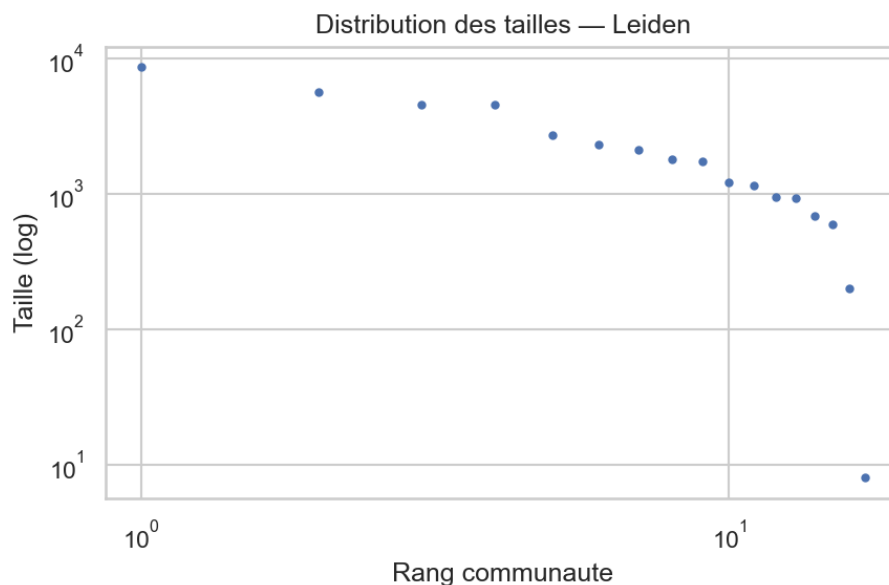


FIGURE 6.1 – Distribution des tailles de communautés Leiden sur le sous-graphe Pokec.

### 6.2 Résultats sur Pokec (40 000 nœuds)

Le tableau 6.1 synthétise les résultats obtenus ( $\gamma = 1$ , Louvain moyenné sur trois graines).

TABLE 6.1 – Comparaison Louvain / Leiden sur le sous-graphe Pokec ( $N = 40\,000$ ).

| Méthode | k  | Q     | Temps   | Connexité         |
|---------|----|-------|---------|-------------------|
| Louvain | 16 | 0,524 | 2,1 min | non garantie      |
| Leiden  | 17 | 0,539 | 7,2s    | 0/17 déconnectées |

**Observations clés :**

- Leiden obtient une modularité légèrement supérieure avec un temps **18 fois plus court** sur cette instance.
- Le NMI mutuel Louvain $\leftrightarrow$ Leiden vaut **0,739** : les partitions sont proches mais non identiques.
- La garantie de connexité de Leiden est vérifiée empiriquement (0 communauté scindée).

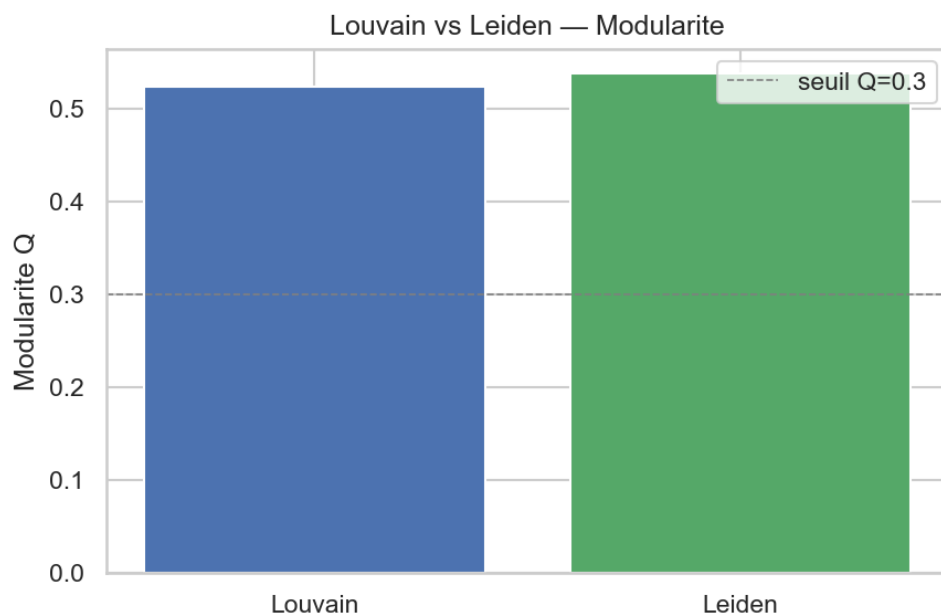


FIGURE 6.2 – Visualisation comparative des partitions Louvain et Leiden.

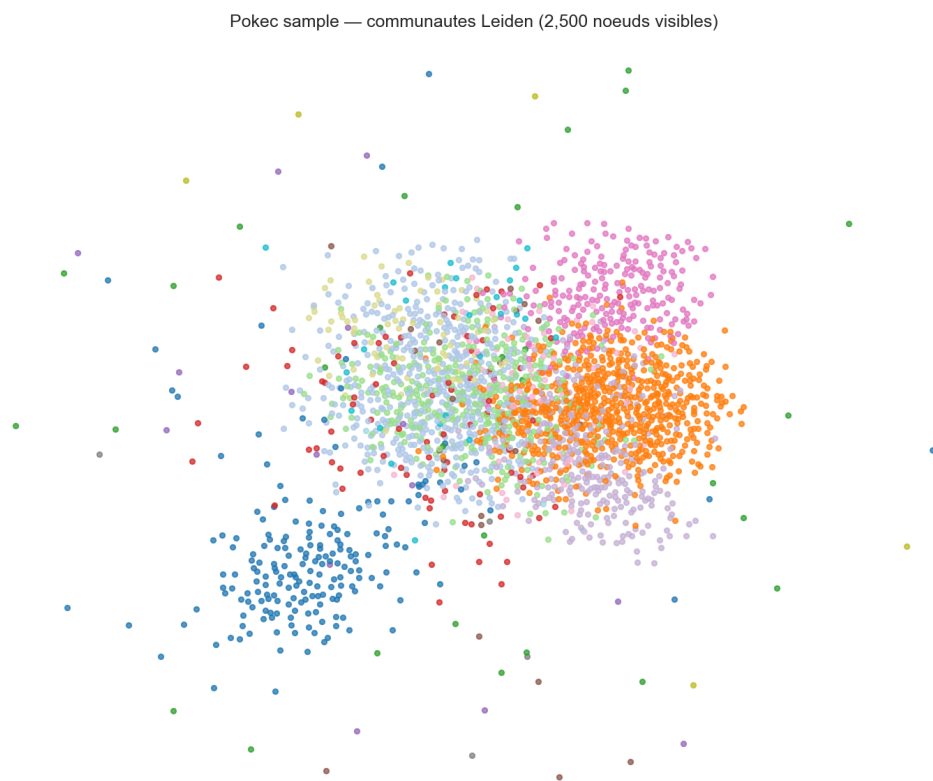


FIGURE 6.3 – Cartographie spring-layout des communautés Leiden (extrait visible).

## 6.3 Validation externe sur YouTube

YouTube fournit environ 5 000 groupes annotés (SNAP). Sur le graphe complet, les deux méthodes modulaires atteignent un  $NMI \approx 0,54$  (tableau 6.2), ce qui traduit un alignement *modéré* avec la vérité terrain — attendu compte tenu de la granularité fine des groupes de référence (médiane  $\approx 3$  membres). L’ARI reste faible ( $\approx 0,06$ ) car l’accord exact sur des milliers de micro-communautés est difficile.

TABLE 6.2 – Validation YouTube sur le graphe complet.

| Méthode | Q     | NMI   | ARI   |
|---------|-------|-------|-------|
| Louvain | 0,648 | 0,540 | 0,061 |
| Leiden  | 0,650 | 0,546 | 0,063 |

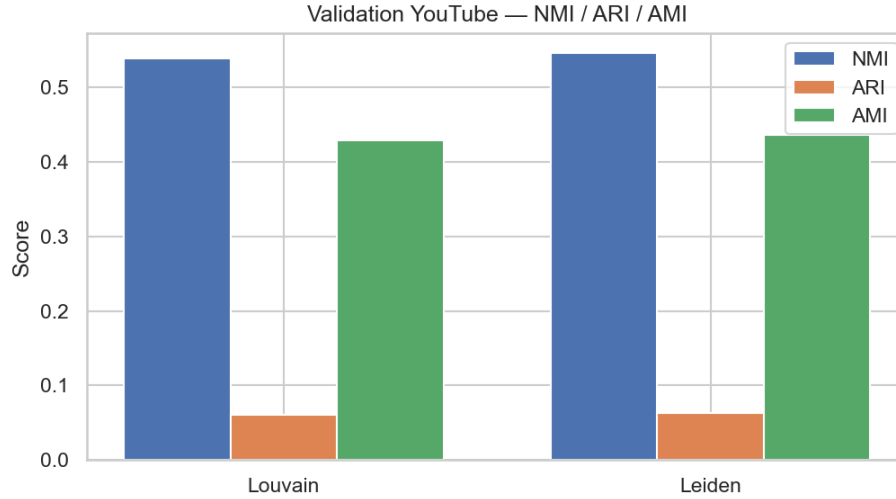


FIGURE 6.4 – Scores NMI, ARI et AMI sur YouTube.

## 6.4 Résultats du GraphSAGE

Avec quatre features structurelles et une tâche de link prediction, le GNN + K-Means produit une partition de qualité modulaire nettement inférieure ( $Q \approx 0,10$  contre  $Q \approx 0,54$  pour Leiden). Ce résultat confirme que, sans attributs riches ni supervision communautaire, l’approche neuronale ne surpasse pas les méthodes modulaires sur  $Q$ .

En revanche, la projection t-SNE des embeddings (figure 6.5) révèle une structure en grappes exploitable pour la visualisation et des analyses qualitatives.

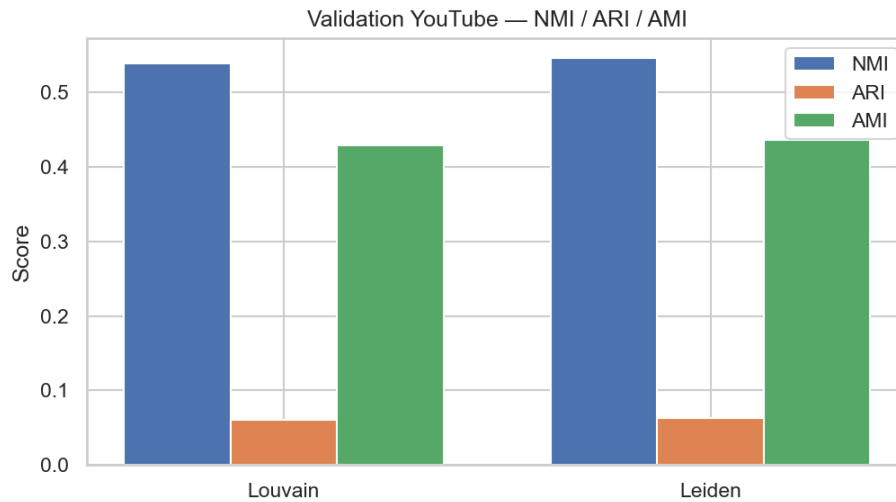


FIGURE 6.5 – Projection t-SNE des embeddings GraphSAGE, colorée par cluster.

## 6.5 Balayage du paramètre $\gamma$

Le paramètre de résolution  $\gamma$  module le nombre de communautés. Sur 50 000 nœuds, un  $\gamma$  faible (0,4) fait exploser  $k$  pour Louvain ( $> 1\,600$  communautés), tandis que Leiden reste plus stable. La valeur  $\gamma = 1$  offre le meilleur compromis  $Q$  / interprétabilité (figure 6.6).

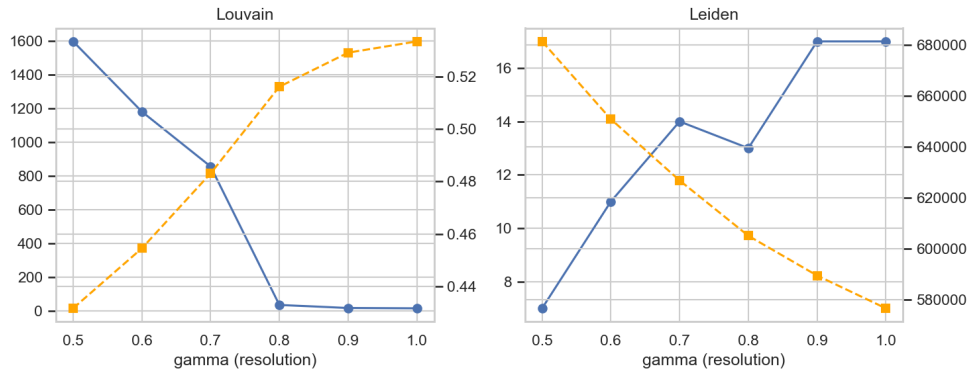


FIGURE 6.6 – Évolution de  $k$  et  $Q$  en fonction de  $\gamma$  (Pokec, 50 000 nœuds).

## 6.6 Synthèse et limites

TABLE 6.3 – Guide de choix méthodologique.

| Besoin                                 | Recommandation               |
|----------------------------------------|------------------------------|
| Prototypage rapide                     | Louvain                      |
| Production (connexité garantie)        | Leiden                       |
| Grand graphe (> 50 k nœuds)            | Leiden + échantillonnage BFS |
| Validation sans GT Pokec               | YouTube (NMI / ARI)          |
| Attributs riches / prédiction de liens | GraphSAGE                    |

### Enseignements :

1. Les graphes sociaux exhibent une structure communautaire détectable automatiquement ( $Q > 0,3$ ).
2. **Leiden** constitue le meilleur compromis qualité–temps–robustesse pour alimenter le produit (recommandation, marketing viral).
3. Le GNN reste une piste complémentaire lorsque des features sémantiques seront intégrées.

**Limites** : échantillonnage BFS (biais vers le hub), absence de ground truth Pokec, features GNN limitées aux centralités, contraintes matérielles sur l'entraînement. Ces points ouvrent des perspectives pour des travaux ultérieurs (Chapitre 7).

# Chapitre 7

## Application d'un système de recommandation basé sur les communautés détectées

### 7.1 Du graphe au produit

Les résultats du Chapitre 6 ne se limitent pas à une évaluation quantitative : la partition Leiden et les centralités calculées au Chapitre 5 alimentent deux fonctionnalités concrètes — **recommandation d'amis** et **marketing viral** — implémentées en Python et exposées via un dashboard Next.js.

L'hypothèse structurante est l'**homophilie** : des utilisateurs d'une même communauté partagent plus souvent des intérêts ou des connexions latentes que des paires inter-communautés. Les centralités (PageRank, betweenness) complètent cette information en identifiant *qui* influence le groupe et *qui* relie plusieurs groupes.

### 7.2 Architecture du pipeline applicatif

Le système repose sur la partition Leiden (référence produit) et les centralités pré-calculées. Trois fonctions Python constituent le cœur applicatif : `recommend_friends()`, `select_viral_seeds()` et `estimate_reach()`.

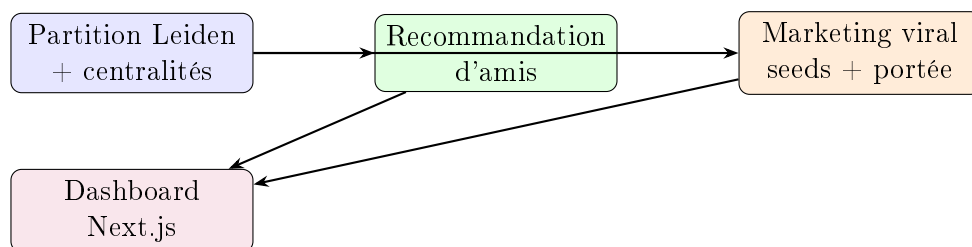


FIGURE 7.1 – Chaîne produit : de la structure du graphe aux applications interactives.

## 7.3 Recommandation d'amis

### 7.3.1 Algorithme `recommend_friends`

La fonction `recommend_friends(user, G, partition, pagerank, top_k)` suit quatre étapes :

1. vérifier que l'utilisateur appartient à une communauté  $C$ ;
2. lister les nœuds de  $C$  non encore connectés à l'utilisateur;
3. classer par PageRank décroissant (proximité sociale + influence intra-groupe);
4. retourner les `top_k` candidats.

Listing 7.1 – Implémentation de `recommend_friends`.

```
1 def recommend_friends(user, G, communities, pagerank, top_k=5):
2     if user not in communities:
3         return []
4     comm = communities[user]
5     friends = set(G.neighbors(user)) if G.has_node(user) else set()
6     candidates = [n for n, c in communities.items()
7                   if c == comm and n != user and n not in friends]
8     return sorted(candidates, key=lambda x: pagerank.get(x, 0),
9                  reverse=True)[:top_k]
```

Le score PageRank ne mesure pas la distance géographique mais la *visibilité* au sein du réseau : un contact recommandé est à la fois pertinent (même communauté) et potentiellement influent.

### 7.3.2 Exemple numérique

L'expérience suivante illustre le comportement de l'algorithme pour l'utilisateur **27 509** (PageRank maximal sur l'extrait analysé), membre de la communauté 0 :

TABLE 7.1 – Top-5 recommandations pour l'utilisateur 27509.

| Rang | Candidat | PageRank | Communauté |
|------|----------|----------|------------|
| 1    | 70 276   | 0,00139  | 0          |
| 2    | 76 928   | 0,00089  | 0          |
| 3    | 108 483  | 0,00085  | 0          |
| 4    | 163 786  | 0,00081  | 0          |
| 5    | 174 520  | 0,00076  | 0          |

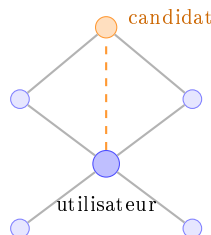


FIGURE 7.2 – Logique de recommandation : lien suggéré (pointillé) vers un membre influent de la même communauté.

La même logique s'étend à la recommandation de contenus populaires *au sein* d'une communauté, sans nécessiter de features sémantiques supplémentaires.

## 7.4 Marketing viral : sélection de seeds

### 7.4.1 Principe et stratégies

Pour diffuser un message, on sélectionne un petit ensemble de **seeds** (comptes de départ) plutôt que de cibler le réseau au hasard. Deux signaux structurels guident le choix :

- **leaders** — PageRank élevé *dans* la communauté (forte audience locale) ;
- **ponts** — betweenness élevée (relient plusieurs communautés).

La fonction `select_viral_seeds()` combine ces signaux selon trois stratégies (tableau 7.2).

TABLE 7.2 – Stratégies de sélection des seeds.

| Stratégie | Score                                                        |
|-----------|--------------------------------------------------------------|
| Influence | PageRank                                                     |
| Ponts     | Betweenness                                                  |
| Équilibré | $0,6 \times \text{PageRank} + 0,4 \times \text{Betweenness}$ |

Par défaut, l'algorithme retient **3 leaders par communauté** et **5 ponts inter-communautés**, soit environ 56 seeds sur 17 communautés Leiden.

### 7.4.2 Estimation de portée

`estimate_reach(seeds, G, hops=2)` propage un BFS depuis les seeds et compte les nœuds atteignables en  $n$  sauts. Sur un extrait de **20 000 nœuds**, **59 seeds** suffisent à couvrir **100 %** du sous-graphe en deux sauts — preuve empirique de l'efficacité d'un ciblage structurel parcimonieux.

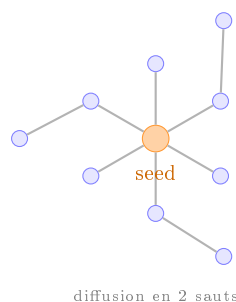


FIGURE 7.3 – Propagation depuis un seed : couverture locale puis extension inter-groupe.

## 7.5 Leaders et nœuds-ponts

L'analyse des centralités par communauté distingue deux rôles complémentaires (figures 7.4 et 7.5).

- **Leader** (ex. nœud 27 509) : degré 1 197, PageRank = 0,0032, communauté 0 — hub local très connecté.
- **Pont** (ex. nœud 71 414) : betweenness = 0,26, communauté 1 — relie des zones du graphe autrement faiblement connectées.

Les listes de leaders et de nœuds-ponts alimentent le dashboard et permettent un ciblage marketing différencié (influence locale *vs.* diffusion transversale).

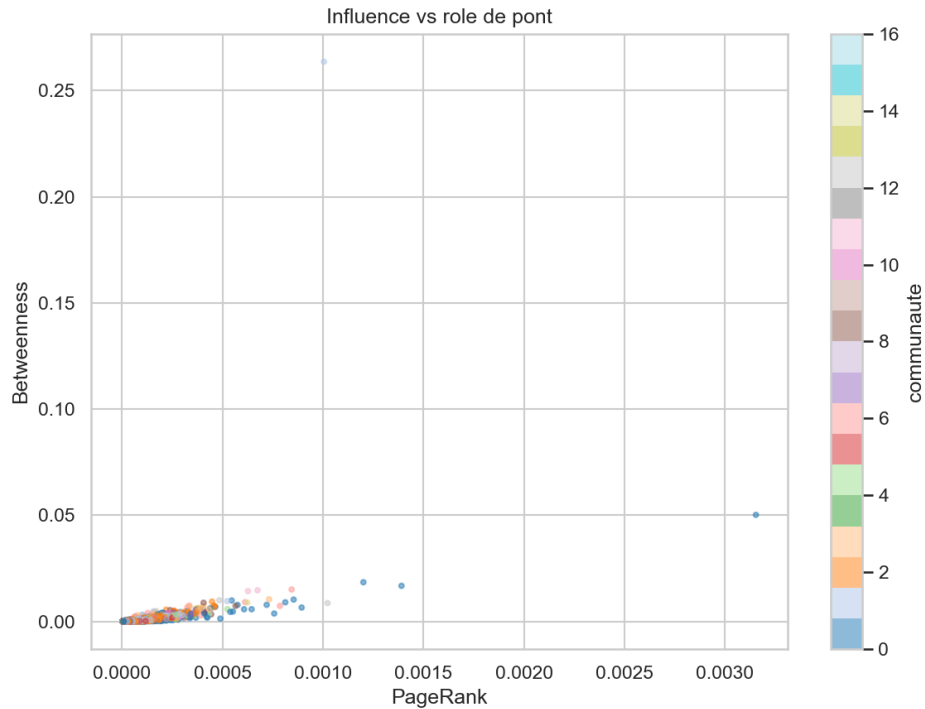


FIGURE 7.4 – Comparaison PageRank et centralité d'intermédierité par communauté.

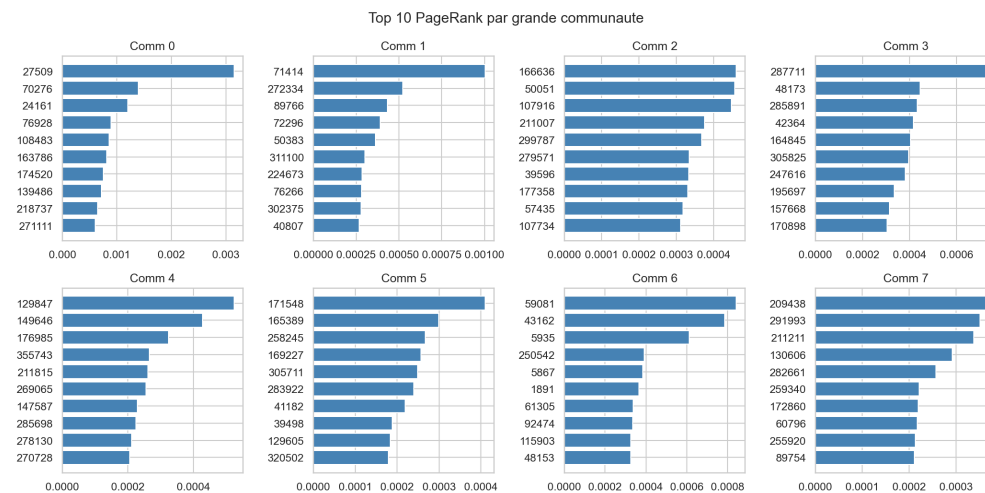


FIGURE 7.5 – Leaders d'opinion identifiés par communauté (PageRank).

## 7.6 Profils communautaires et segmentation

Au-delà de la structure pure, les profils agrégés relient chaque cluster à des attributs démographiques. Exemple pour la communauté 0 (8 699 membres) : région dominante *zilinsky kraj* (16 %), répartition genre 51 %/49 %. Ces profils permettent une segmentation marketing interprétable — chaque communauté Leiden devient un segment d’audience potentiel.

## 7.7 Dashboard interactif

Le dashboard Next.js charge un échantillon exploratoire de **800 nœuds** (1 324 arêtes) pour une navigation fluide dans le navigateur, tout en conservant les KPIs calculés sur l’échantillon complet de 20 000 utilisateurs.

TABLE 7.3 – Pages du dashboard et fonctionnalités associées.

| Route      | Fonction                                                          |
|------------|-------------------------------------------------------------------|
| /          | Vue d’ensemble, KPIs Pokec, comparaison Karate                    |
| /recommend | <code>recommend_friends()</code> interactif                       |
| /viral     | <code>select_viral_seeds()</code> + <code>estimate_reach()</code> |
| /explore   | Exploration pédagogique (Karate Club)                             |
| /compare   | Comparaison Louvain / Leiden / GNN                                |



FIGURE 7.6 – Chaîne de déploiement du pipeline vers le dashboard.

### 7.7.1 Vue d’ensemble (/)

La page d’accueil synthétise le cas d’usage Pokec : 20 000 utilisateurs, 18 communautés Leiden, 59 seeds virales et le hub par défaut (nœud 27 509). Elle rappelle le pipeline en quatre étapes et propose un accès direct aux modules recommandation, viral et démo Karate. Un panneau comparatif affiche les métriques Louvain / Leiden / GNN sur le graphe de Zachary (figure 7.7).

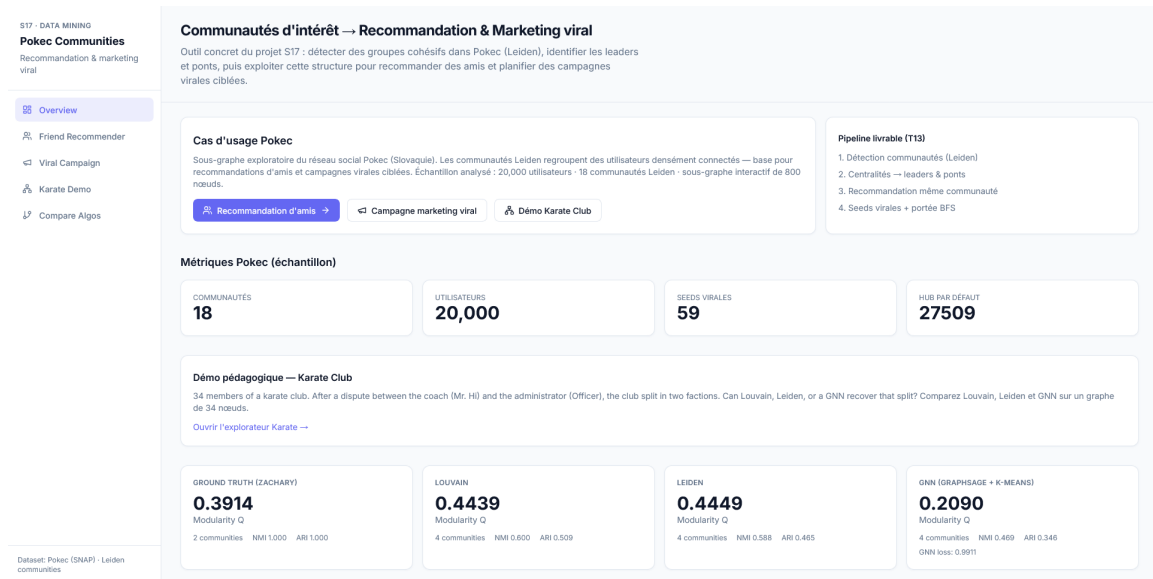


FIGURE 7.7 – Page / du dashboard : vue d'ensemble et KPIs du projet.

## 7.7.2 Recommandation d'amis (/recommend)

L'utilisateur saisit un identifiant Pokec et le paramètre `top-k`. Le dashboard affiche le profil (communauté, degré, PageRank, région dominante) et la liste des candidats non connectés classés par influence. La visualisation force-directed met en évidence l'utilisateur cible et ses recommandations au sein du cluster (figure 7.8).

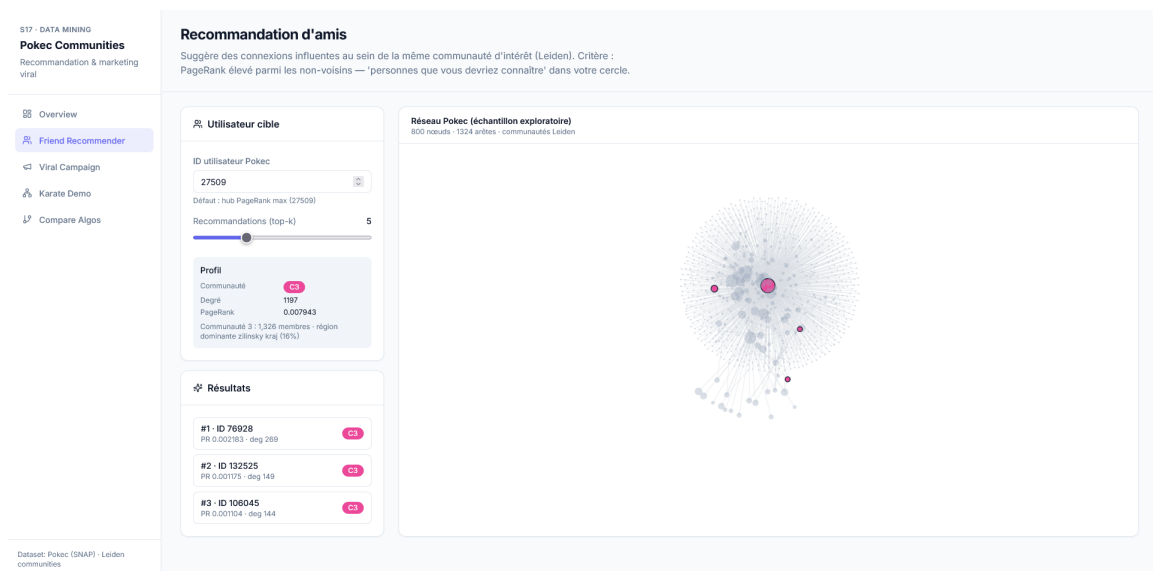


FIGURE 7.8 – Page /recommend : recommandation interactive pour l'utilisateur 27509.

## 7.7.3 Marketing viral (/viral)

Cette page permet de choisir la stratégie de seeds (influence, ponts ou équilibré), le nombre de leaders par communauté et les ponts inter-communautés. La portée BFS en  $n$  sauts est recalculée en temps réel : avec 54 seeds (49 intra + 5 ponts) et 2 sauts,

l'échantillon de 800 nœuds est couvert à 100 %. Le tableau des profils communautaires (région, genre) guide le ciblage marketing (figure 7.9).

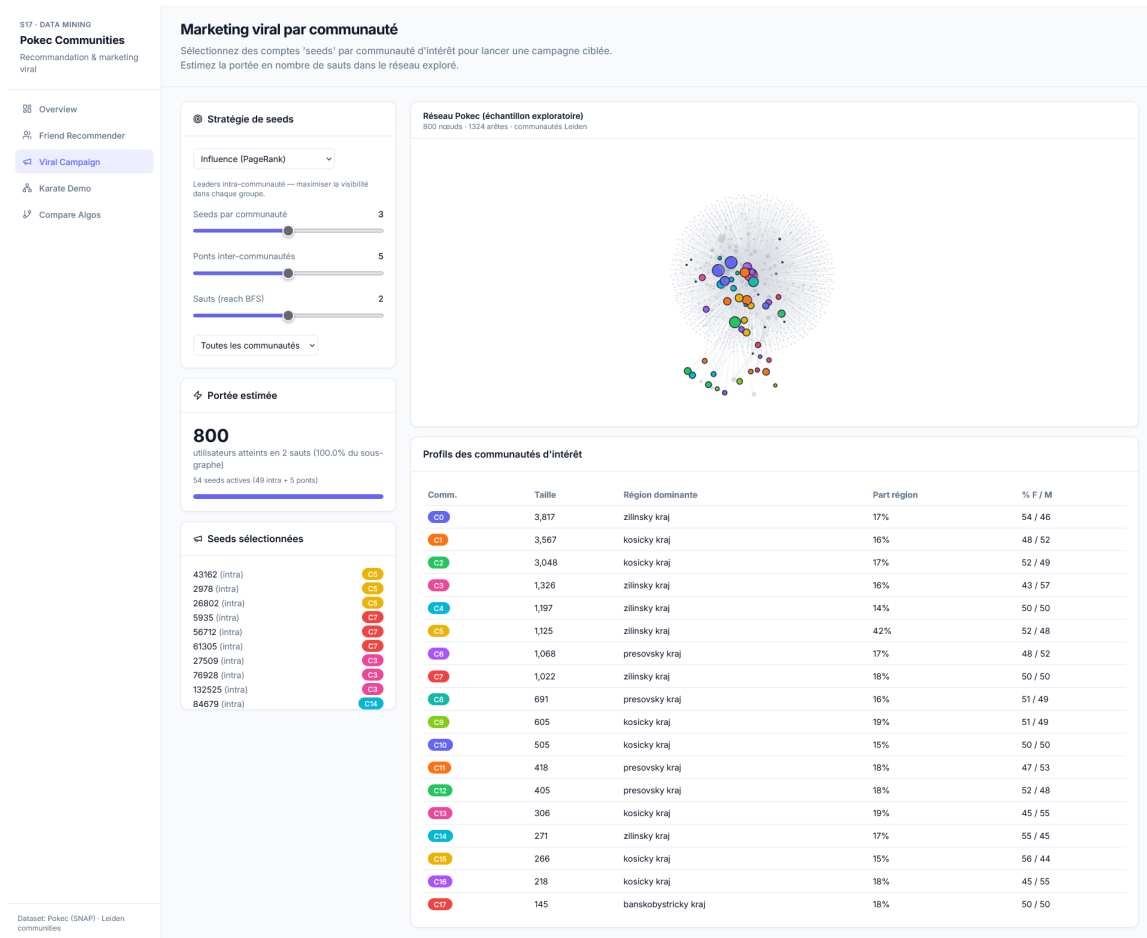


FIGURE 7.9 – Page /viral : sélection de seeds et estimation de portée.

## 7.8 Perspectives

Plusieurs extensions sont envisageables sans remettre en cause l'architecture actuelle :

- intégration d'attributs sémantiques (intérêts, publications) dans le score de recommandation ;
- détection dynamique des communautés sur flux de nouvelles arêtes ;
- couplage Leiden + GraphSAGE pour un scoring hybride structure-embedding ;
- déploiement sur le graphe Pokec complet ( $\approx 1,6$  M nœuds) avec partitionnement distribué.

Le projet démontre qu'une chaîne KDD complète — du graphe brut à un outil de recommandation et de marketing viral — peut être construite autour de la détection de communautés Leiden et des mesures de centralité classiques.

# Conclusion générale

Ce rapport a porté sur la **détection de communautés** dans les graphes sociaux et sur l'exploitation concrète des partitions obtenues pour la recommandation d'amis et le marketing viral. À partir des réseaux *Pokec* et *YouTube*, nous avons mis en œuvre un pipeline de fouille de données (KDD) comparant trois familles d'approches : les méthodes modulaires **Louvain** et **Leiden**, ainsi qu'une piste **GraphSAGE** couplée à un clustering.

**Parcours méthodologique.** Après avoir posé les fondements théoriques (Chapitre 1) et décrit le prétraitement des graphes massifs — échantillonnage BFS, filtrage par degré, calcul des centralités (Chapitre 2) — nous avons formalisé la modularité  $Q$  comme critère d'évaluation interne (Chapitre 3), puis détaillé les algorithmes étudiés et leurs compromis qualité-temps-robustesse (Chapitre 4). L'implémentation Python, la validation sur YouTube et le déploiement via le dashboard constituent le cœur expérimental (Chapitres 5 et 6).

**Principaux enseignements.** Sur un sous-graphe Pokec de **40 000 nœuds**, **Leiden** surpasse Louvain sur les trois axes mesurés : modularité légèrement supérieure ( $Q = 0,539$  contre  $0,524$ ), temps d'exécution nettement inférieur ( $7,2\text{s}$  contre  $2,1\text{min}$ ) et garantie de connexité des communautés ( $0/17$  déconnectées). L'accord mutuel entre les deux méthodes modulaires reste élevé ( $\text{NMI} \approx 0,74$ ), ce qui confirme la stabilité de la structure détectée. La validation externe sur YouTube ( $\text{NMI} \approx 0,54$ ,  $\text{ARI} \approx 0,06$ ) traduit un alignement modéré avec des milliers de micro-groupes annotés — résultat cohérent avec la granularité fine de la vérité terrain. En revanche, le GNN, entraîné sans supervision communautaire et avec des features limitées aux centralités structurelles, n'atteint qu'une modularité d'environ  $0,10$  et ne surpasse pas les approches modulaires sur ce critère.

**Du résultat au produit.** L'apport distinctif du projet réside dans la **valorisation opérationnelle** des partitions (Chapitre 7). Les fonctions `recommend_friends()`, `select_viral_seeds` et `estimate_reach()` transforment la structure du graphe en recommandations ciblées et en campagnes virales parcimonieuses. Sur l'échantillon exploratoire, une cinquantaine de seeds suffit à couvrir l'intégralité du sous-graphe en deux sauts de propagation. Le dashboard Next.js matérialise ces résultats à travers trois pages clés : vue d'ensemble (/), recommandation interactive (/recommend) et planification virale (/viral).

**Limites et perspectives.** Plusieurs limites doivent être gardées à l'esprit : l'échantillonnage BFS introduit un biais vers le voisinage du hub de départ ; Pokec ne dispose pas de vérité terrain communautaire, ce qui impose une validation indirecte ; le GNN n'exploite pas encore les attributs sémantiques riches du réseau ; enfin, le partitionnement du graphe complet ( $\approx 1,6\text{M}$  nœuds) dépasse les ressources d'une exécution locale standard.

Les perspectives naturelles consistent à enrichir les scores de recommandation avec des attributs utilisateurs, à explorer des approches hybrides Leiden–GNN, à mettre à jour les communautés de façon dynamique et à porter le pipeline sur une infrastructure distribuée.

**Mot de fin.** En synthèse, ce travail démontre qu’une chaîne KDD complète — de l’exploration d’un réseau social massif à un outil interactif de recommandation et de marketing viral — est réalisable avec des méthodes éprouvées. **Leiden** s’impose comme la référence pour la production, **Louvain** demeure pertinent pour le prototypage rapide, et les réseaux de neurones sur graphes constituent une piste complémentaire lorsque des features riches et une supervision adaptée seront disponibles. Au-delà de la comparaison algorithmique, le projet illustre comment des communautés d’intérêt automatiquement détectées peuvent devenir un levier concret pour la personnalisation et la diffusion ciblée dans les réseaux sociaux.

# Bibliographie

- [1] Fayyad U., Piatetsky-Shapiro G., Smyth P. From data mining to knowledge discovery in databases. *AI Magazine*, 17(3) :37–54, 1996.
- [2] Blondel V.D., Guillaume J.L., Lambiotte R., Lefebvre E. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics*, 2008.
- [3] Traag V.A., Waltman L., Van Eck N.J. From Louvain to Leiden : guaranteeing well-connected communities. *Scientific Reports*, 2019.
- [4] Khettaf D., Djenouri D., Rezaeifar Z., Djenouri Y. Hybrid Graph Embeddings and Louvain Algorithm for Unsupervised Community Detection. 2025.